Problema de optimizare neconstransa in invatarea aprofundata

Dinu Eduard Alexandru

Aprilie 2026

Contents

1 Cerinte **1**

2 Baza de date **2**

3 Retea **4**
3.1 Forward propagation 5
3.2 Backward propagation 6

4 Algoritmi de optimizare **7**
4.1 Gradient Stocastic cu loturi simplu (SGD) 7
4.2 Gradient Stocastic cu loturi si inertie (SGDM) 8
4.3 Adaptive Moment Estimation (Adam) 8

5 Rezultatele algoritmilor **9**

6 Anexa 1 **13**

7 Anexa 2 **13**

1 Cerinte

- Selectati o baza de date adecvata invatarii supervizate (i.e. problema clasificarii sau regresiei). Exemple de site-uri cu baze de date: IC Irvine Machine Learning Repository, Kaggle datasets etc. Baza de date ar trebui sa aiba cel putin 100 de exemple (din care 80% pentru antrenare si 20% pentru testare).
- Stabiliti un numar de neuroni $m \geq 10$ pentru stratul ascuns si folositi functia de activare din sectiunea 4, conform tabelului de pe moodle.
- Rezolvati problema de optimizare (1) implementand cel putin 2 metode de optimizare de la curs.

- Comparati metodele alese in termeni de timp si iterati analizand norma gradientului, functia obiectiv de-a lungul iteratiilor si timp.
- Analizati performanta sarcinii de invatare alesa (clasificare sau regresie) folosind metrici potrivite cum ar fi precizia clasificatorului (matrice de confuzie), coeficientul de determinare R_2 etc.

Ca sarcina am ales clasificarea binara cu functia de loss de entropie incrucisata binara:

$$L(e, y) = -\frac{1}{N}\sum_{i=1}^{N}(e_i log\, y_i + (1 - e_i)\ log(1 - y_i))$$

Functia de activare folosita a fost TSReLU, cu forma si derivata:

$$\sigma(z) = \frac{1}{(1 + e^{-z})}$$

$$TSReLU(z) = z \tanh(\sigma(z))$$

Derivata:

$$TSReLU(z)' = tanh(\sigma(z)) + z(1 - tanh(\sigma(z))^2)\sigma(z)(1 - \sigma(z))$$

2 Baza de date

Baza de date selectata este Predict Students' Dropout and Academic Success, contine 36 de caracteristici si 4424 instante in total, 3630 dupa curatarea datelor.

Dupa o impartire de 80%-20% ramanem cu:

- 2904 instante pentru antrenare
- 726 instante pentru testarea finala

1. Marital Status
2. Application mode
3. Application order
4. Course
5. Daytime/evening attendance
6. Previous qualification
7. Previous qualification (grade)
8. Nationality

9. Mother's qualification
10. Father's qualification
11. Mother's occupation
12. Father's occupation
13. Admission grade
14. Displaced
15. Educational special needs
16. Debtor
17. Tuition fees up to date
18. Gender
19. Scholarship holder
20. Age at enrollment
21. International
22. Curricular units 1st sem (credited)
23. Curricular units 1st sem (enrolled)
24. Curricular units 1st sem (evaluations)
25. Curricular units 1st sem (approved)
26. Curricular units 1st sem (grade)
27. Curricular units 1st sem (without evaluations)
28. Curricular units 2nd sem (credited)
29. Curricular units 2nd sem (enrolled)
30. Curricular units 2nd sem (evaluations)
31. Curricular units 2nd sem (approved)
32. Curricular units 2nd sem (grade)
33. Curricular units 2nd sem (without evaluations)
34. Unemployment rate
35. Inflation rate
36. GDP

37. Target

In Target initial erau 3 tipuri de posibile situatii, Dropout, Graduate si Enrolled, acestea 3 formau un total de 4424 campuri valide, am ales sa elimin campurile de tipul Enrolled, reducand numarul de campuri la 3630
Scopul este de a determina daca un individ a terminat sau nu a terminat facultatea in functie de situatia sa.

3 Retea

Reteaua neuronala este formata din 3 layere, 1 layer de intrare cu 37 de neuroni (1 neuron pentru fiecare caracteristica (total 36) + 1 neuron de tip bias)

Iesire este formata dintr-un singur neuron pentru clasificarea binara cu prag de 0.5 si functia de activare Sigmoid

Stratul ascuns este format din 14 neuroni cu functia de activare TSReLU

Propagarea prin retea este impartita in 2 parti:

1. Forward propagation
2. Backward propagation

3.1 Forward propagation

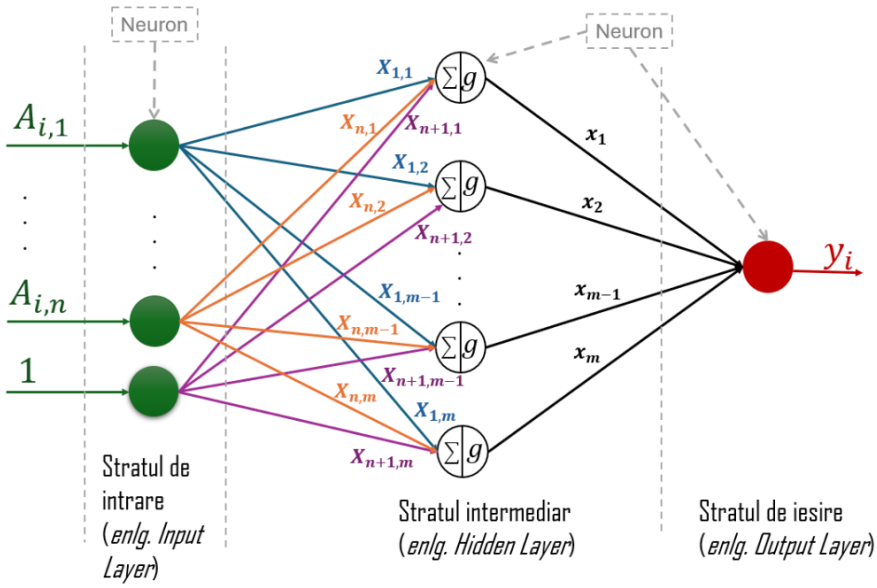

Figure 1: Exemplu de retea superficiala: n intrati, m neuroni pe stratul ascuns si o iesire

Modelul de predictie ce rezulta din Figura 1 este:

$$Z = AW,$$

$$H = TSReLU(Z)$$

$$u = Hw$$

In plus eu mai aplic o functie de activare la iesire $\sigma(u) = \frac{1}{1+e^{-u}}$ adica iesirea din retea devine:

$$\hat{y} = \sigma(u)$$

Unde:

- Z = intrarea in stratul ascuns
- A = matricea de intrare
- W = greutatile dintre stratul de intrare si stratul ascuns

- H = iesirea stratului ascuns
- u = intrarea in ultimul strat
- w = greutatile dintre stratul ascuns si iesire
- $\hat{y}$ = iesirea din retea dupa functia de activare, sub forma de procent

3.2 Backward propagation

Gradientul parametrilor retelei este calculat prin backpropagation, adica forward propagation, dar invers, adica propagarea inversa a erorii.

Eroarea de iesire din retea este:

$$\delta^{(out)} = \frac{\partial L}{\partial u}$$

Dar, inmultirea celor 2 derivate este: $\frac{\partial L}{\partial u} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial u}$

Daca le calculam:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

$$\frac{\partial \hat{y}}{\partial u} = \hat{y}(1-\hat{y})$$

$$\frac{\partial L}{\partial u} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \cdot \hat{y}(1-\hat{y})$$

$$\frac{\partial L}{\partial u} = \hat{y} - y$$

Dar, deoarece folosesc functia de iesire $\sigma(u)$ si cross-entropy pentru functia de pierdere, eroarea la iesire poate fi simplificata in:

$$\delta^{(out)} = \hat{y} - y$$

Asadar, gradientul lui w poate fii scris ca:

$$\nabla w = \frac{1}{N} H^T \delta^{(out)}$$

Iar propagarea pe stratul ascuns este:

$$\delta^{(hid)} = (\delta^{(out)} w^T) \odot TSReLU'(Z)$$

Cu gradientul lui W rezultat fiind:

$$\nabla W = \frac{1}{N} A^T \delta^{(hid)}$$

Unde:

- $\delta^{(out)}$ = Eroarea la iesirea retelei
- y = iesirea corecta (Eticheta)
- $\hat{y}$ = iesirea din retea dupa functia de activare, sub forma unei probabilitati
- H = iesirea stratului ascuns
- TSReLU' = derivata functiei de activare
- A = datele de intrare
- Z = intrarea in stratul ascuns
- N = numarul de exemple din lot

4 Algoritmi de optimizare

Pentru antrenarea retelei neuronale am ales 2 algoritmi similari de tipul Metodei Gradient Stocastica si o implementare a algoritmului ADAM.

1. Gradient Stocastic cu loturi simplu
2. Gradient Stocastic cu loturi si inertie
3. Adam (Adaptive Moment Estimation)

4.1 Gradient Stocastic cu loturi simplu (SGD)

- Se alege un numar de n intrari la fiecare iteratie, denumita si lot.
- Se antreneaza clasic, pentru fiecare lot, se calculeaza gradientul functiei de pierdere prin **backwardsProp** si se ajusteaza parametrii (greutatiile conexiunilor) in directia opusa gradientului
- Aceasta abordare introduce zgomot in actualizarile dar, ajuta la minimizarea functiei loss si evita unele minime locale slabe
- Forma generala a algoritmului:

  $$\theta_{k+1} = \theta_k - \alpha \nabla \mathcal{L}_{B_k}(\theta_k)$$

  Unde:

  - θ_k = parametrii la pasul k
  - α = rata de invatare sau pas
  - B_k = lotul curent
  - $\nabla \mathcal{L}_{B_k}(\theta_k)$ = gradientul lotului la pasul k
- Avantaj: simplu, cost scazut pe iteratie
- Dezavantaj: convergenta lenta si zgomot

4.2 Gradient Stocastic cu loturi si inertie (SGDM)

- Metoda SGD cu inertie sau moment, SGD, extinde SGD prin introducerea inertiei in ajustarea parametrilor
- In loc sa mearga in directia gradientului, acesta combina directiile anterioare cu cea curenta, acest lucru duce la o miscare mai stabila si mai rapida.
- In acest mod, oscilatiile sunt reduse iar convergenta este atinsa mai rapid decat in SGD simplu.
- Forma generala a algoritmului:

$$v_k = \beta v_{k-1} - \alpha \nabla \mathcal{L}_{B_k}(\theta_k)$$

$$\theta_{k+1} = \theta_k - v_k$$

  Unde:

  - v_k = inertia (suma ponderata a gradientelor trecute)
  - β = cat de mult contribuie inertia in sistem
  - α = rata de invatare sau pas
  - θ_k = parametrii la pasul k
  - B_k = lotul curent
  - $\nabla \mathcal{L}_{B_k}(\theta_k)$ = gradientul lotului la pasul k
- Avantaj: convergenta mai stabila si rapida
- Dezavantaj: depinde foarte mult de coeficientul de inertie

4.3 Adaptive Moment Estimation (Adam)

- Adam combina metoda SGD cu inertie cu o alta metoda RMSProp.
- In RMSProp se foloseste de RMS (Real mean square) a gradientului pentru a ajusta pasul, adica o panta mare, pas mic; panta mica (un platou de exemplu) gradient mic, RMS mic, deci si un pas mare
- Daca combinam inertia cu RMS putem obtine o combinatie buna ce ne poate ajuta sa evitem minimele false si de a converge mult mai rapid.
- Pe scurt, momentul reduce zgomotul, mentine directia generala si elimina variatile mici, iar RMS accelereaza pasul in platouri si incetineste in zone abrupte

- Forma generala a algoritmului:

  Momentul 1 (Momentum/Inertia):

$$m_k = \beta_1 m_{k-1} + (1-\beta_1)\nabla\mathcal{L}_{B_k}(\theta_k)$$

  Momentul 2 (RMS):

$$v_k = \beta_2 v_{k-1} + (1-\beta_2)(\nabla\mathcal{L}_{B_k}(\theta_k))^2$$

  Corectii bias:

$$\hat{m}_k = \frac{m_k}{1-\beta_1^k}, \qquad \hat{v}_k = \frac{v_k}{1-\beta_2^k}$$

  Update:

$$\theta_{k+1} = \theta_k - \alpha\frac{\hat{m}_k}{\sqrt{\hat{v}_k}+\epsilon}$$

  Unde:

  - β_1 = cat de mult contribuie inertia in sistem
  - β_2 = cat de mult contribuie RMS in sistem
  - α = rata de invatare sau pas
  - v_k = inertia (suma ponderata a gradientelor trecute)
  - θ_k = parametrii la pasul k
  - B_k = lotul curent
  - $\nabla\mathcal{L}_{B_k}(\theta_k)$ = gradientul lotului la pasul k
  - ϵ = o variabila infinitezimal de mica, in codul meu este 10^{-8} sau 1e-8

- Avantaj: converge rapid si este mai adaptiv

- Dezavantaj: mai scump comparativ cu primele 2 metode, poate sa fie identic sau mai slab decat SGDM in anumite cazuri, are mai multe variabile in algoritm ($\alpha,\ \beta_1,\ \beta_2$)

5 Rezultatele algoritmilor

Toti cei 3 algoritmi au obtinut un scor F1 $\geq$ 90%

Pentru 10000 iteratii am avut urmatoarele rezultate:

Timp de antrenare:

- SGD: 10000 iteratii, 4.197812

- SGDM: 10000 iteratii, 4.709932

- ADAM: 4846 iteratii, 3.716908s

Model	Recall	Precision	Specificity	Accuracy	F1
SGDscores	0.8980	0.8635	0.7673	0.8485	0.8804
SGDMscores	0.9490	0.9049	0.8364	0.9063	0.9264
ADAMscores	0.9579	0.9095	0.8436	0.9146	0.9330
Mean	0.9350	0.8926	0.8158	0.8898	0.9133

Figure 2: Scoruri dupa o rulare

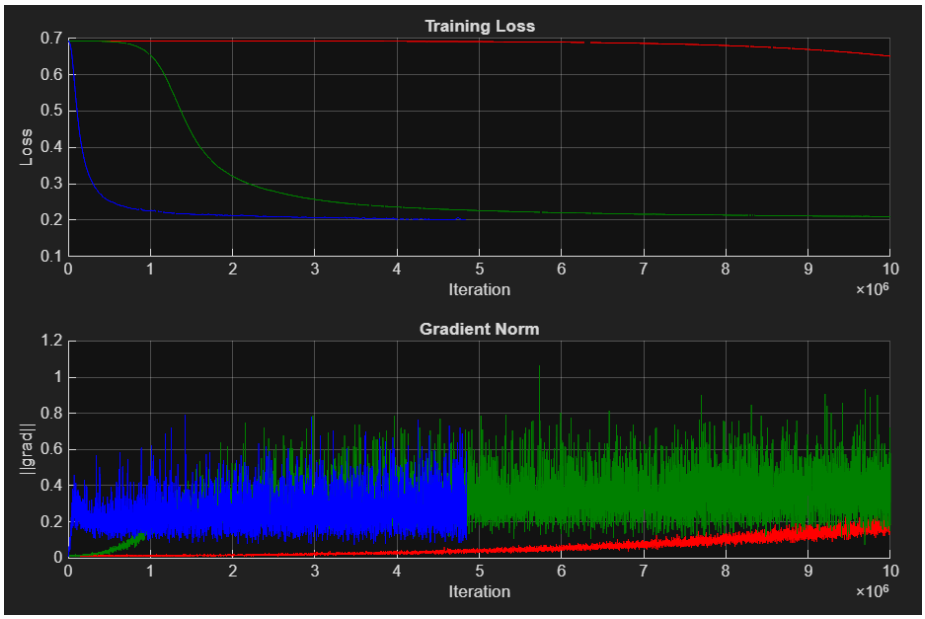

Figure 3: Evolutia pierderii si Normei gradientului

- SGD este rosu
- SGDM este verde
- ADAM este albastru

Se poate vedea cu usurinta ca SGD a ajuns la iteratia 10000 cu un loss de 0.65
SGDM a ajuns la iteratia 10000 cu un loss de 0.21
Adam a ajung la iteratia 4846 cu un loss de 0.2

Toate cele 3 Norme sunt destul de haotice, acest lucru este datorata loturilor, fiecare are un gradient calculat diferit.
In plus, se observa ritmul de schimbare al fiecarui algoritm, ADAM se misca cel mai rapid iar SGD cel mai incet

Matricele de confuzie prezentate mai jos arata ca toti cei trei algoritmi au performante bune, au o directie corecta a clasificarii.

Majoritatea predictiilor sunt corecte, ceea ce sugereaza o acuratete ridicata a modelelor.

Se pot observa diferente intre cei trei algoritmi in ceea ce priveste performanta si valorile functiei de pierdere.

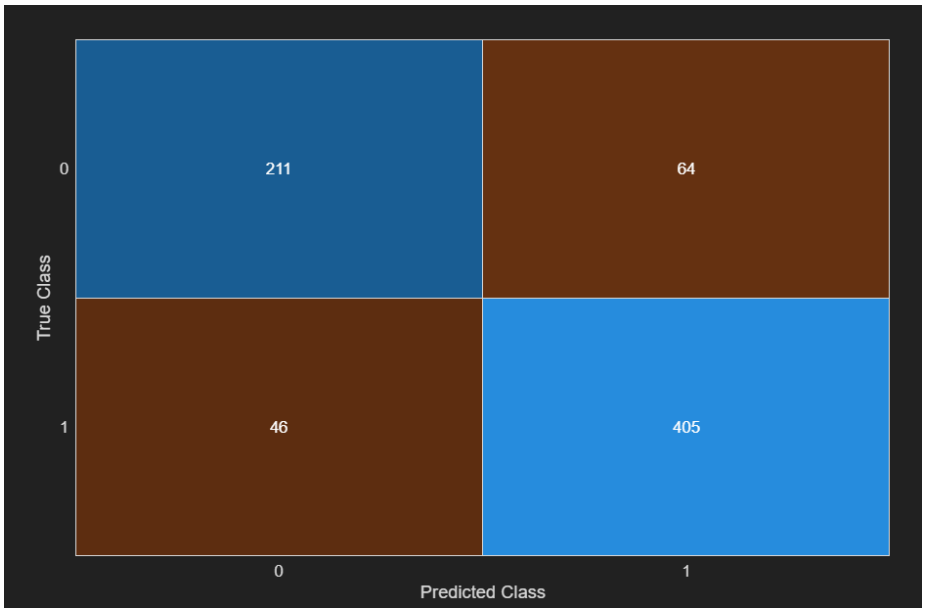

Figure 4: Matrice de confuzie pentru SGD

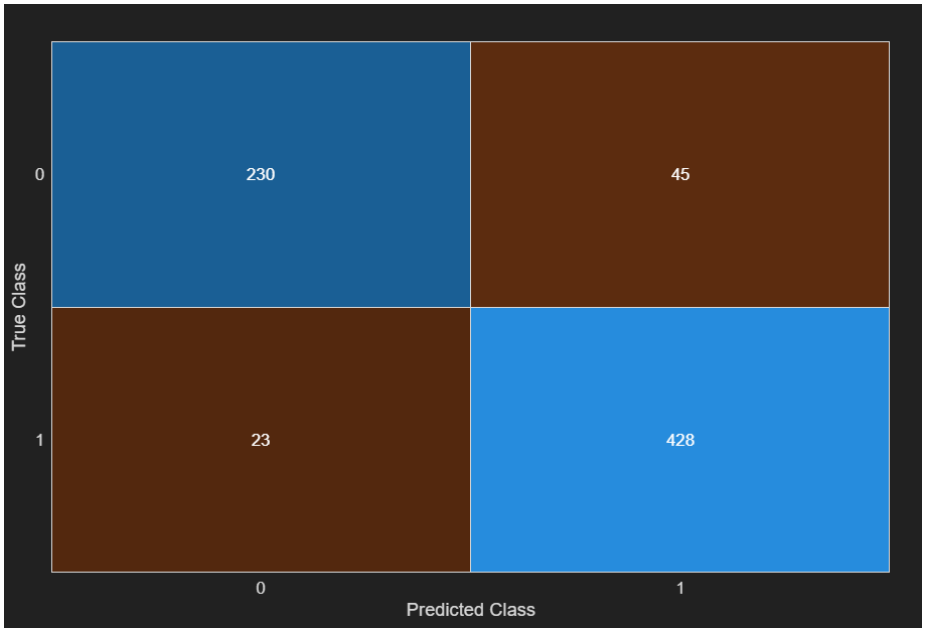

Figure 5: Matrice de confuzie pentru SGDM

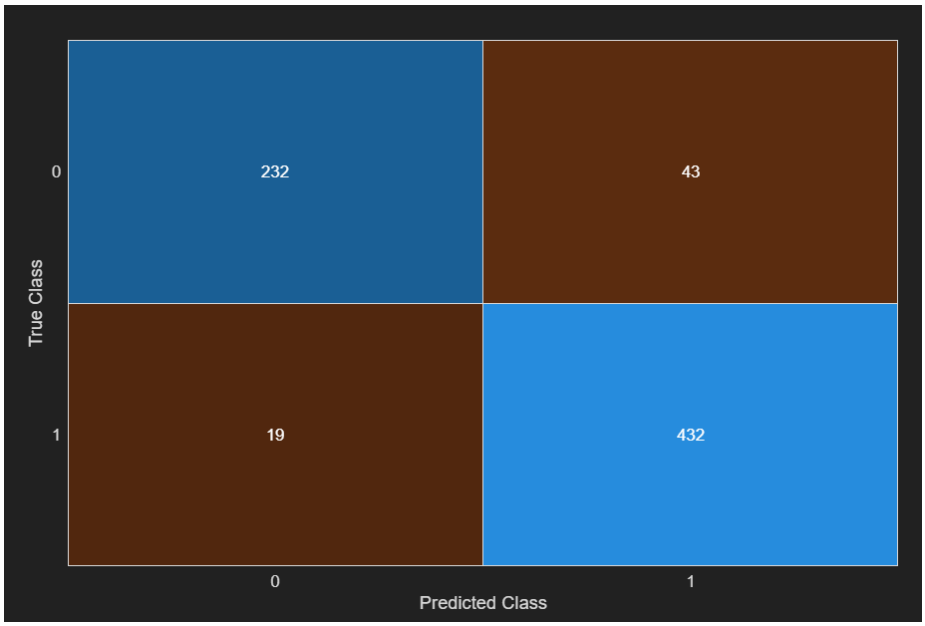

Figure 6: Matrice de confuzie pentru ADAM

6 Anexa 1

Link catre github unde se afla codul si pdf-ul acesta
https://github.com/edi3d3d/Neural-Network

7 Anexa 2

In rezolvarea temei am folosit inteligenta artificiala (ChatGPT) pentru redactarea unor portiuni din raport si in codul matlab.
In raport am validat prim a ma uita daca imi place ce zicea sau nu, iar in matlab am validat prim aceasi metoda + am verificat daca ruleaza. Link catre conversatii (schimb conversatiile des deoarece nu doresc sa se contamineze cu alte informatii):

- https://chatgpt.com/share/69e0ac46-77bc-8397-828e-0f10c25c74a8
- https://chatgpt.com/share/69e0acdb-9c2c-838c-bc4b-d2ffbda5bc4a
- https://chatgpt.com/share/69e0acef-3b88-8395-ba79-a17bf50fee93
- https://chatgpt.com/share/69e0acfd-b278-838f-a5b3-2a0c42b51719
- https://chatgpt.com/share/69e0ad47-9c88-8396-9a97-5e96d876c7ad

- https://chatgpt.com/share/69e0ad5e-1828-8396-b6ae-dea1ba54a919
- https://chatgpt.com/share/69e0ad6f-32f4-8384-997d-41d96cb22318